

7-3 · Embedding과 Vector DB · 이론 1H

임베딩 원리와 유사도 검색

글자가 아니라 뜻으로 찾는 검색 — 벡터·차원·코사인

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 임베딩이 텍스트를 어떤 숫자로 바꾸는지, 차원이 무엇인지 설명한다
- ② 코사인 유사도로 두 문장의 의미적 가까움을 계산한다
- ③ COSINE·EUCLID·DOT 세 거리 기준을 구별한다
- ④ 컬렉션·포인트·payload 를 관계형 DB 용어에 대응시킨다

다음 1H 실습에서 임베딩을 직접 만들어 차원과 유사도를 손으로 확인합니다

1. 키워드 검색과 무엇이 다른가

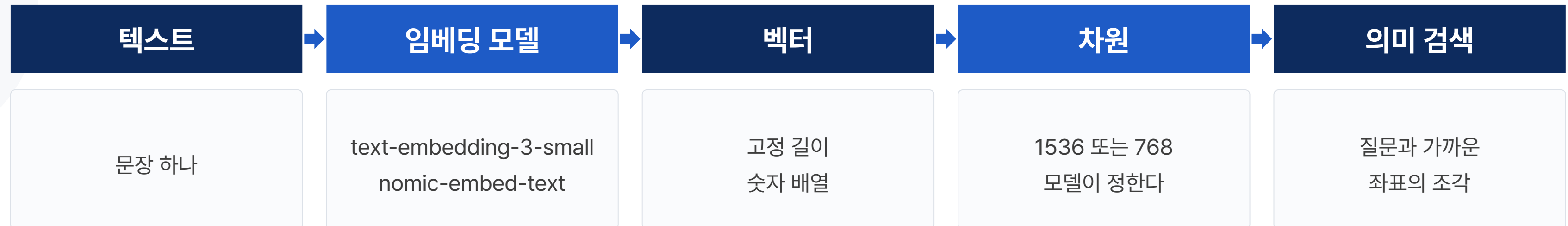
글자가 겹치는가 vs 뜻이 가까운가

항목	키워드 검색	의미(벡터) 검색
찾는 기준	글자가 겹치는가	뜻이 가까운가
「작업 순서 관리 도구」	Airflow 문서를 못 찾음	Airflow 문서를 찾음
오타·유의어	약함	비교적 강함
정확한 코드·품번	강함	오히려 약할 수 있음
속도·비용	가볍다	임베딩 비용이 든다

실무에서는 둘을 함께 쓴다(하이브리드 검색) — 제품 코드·법조문 번호 같은 것은 키워드 검색이 정확

2. 임베딩 벡터

임베딩 — 문장마다 지도 위에 좌표를 찍는 일



같은 모델로 만든 벡터끼리만 비교할 수 있다 — 모델이 다르면 좌표계가 다르다

내용이 비슷한 문장일수록 가까운 자리에 — 실제 좌표는 2차원이 아니라 수백~수천 차원

2. 임베딩 벡터

차원은 모델이 정한다 — 크면 세밀하지만 비싸다

1

임베딩 벡터

텍스트 하나를 고정 길이의 숫자 배열로 바꾼 것

2

차원(dimension) = 배열의 길이

모델이 정한다 — text-embedding-3-small 1536 · nomic-embed-text 768

3

차원은 「표현의 세밀함」

크면 미묘한 차이를 담지만 저장·계산 비용이 늘어난다

모델을 바꾸면

컬렉션 새로

전량 재적재

모델을 바꾸면 이미 저장한 벡터가 전부 무의미해진다 — 실무에서 모델 교체가 큰일인 이유

3. 코사인 유사도

코사인 유사도 — 방향만 보고 길이는 무시한다

1 -1 ~ 1 사이의 값
1에 가까울수록 같은 뜻, 0이면 무관, -1이면 반대 방향

2 길이(크기)는 무시, 방향만
그래서 문장 길이 차이에 덜 흔들린다

3 $\cos([1, 0], [2, 0]) = 1.0$
길이를 2배로 늘려도 값은 같다

4 $\cos([1, 0], [0, 1]) = 0.0$
직각 = 무관

$\text{dot} / (\text{na} \times \text{nb})$ — 내적을 두 벡터 길이의 곱으로 나눈다 · 실습 3단계에서 이 네 줄을 직접 짤다

거리 기준 세 가지 — 오늘은 COSINE

COSINE — 방향

01

텍스트 임베딩의
기본 선택

EUCLID — 직선거리

02

좌표의 절대 위치가
의미 있을 때

DOT — 내적

03

벡터 길이 자체에
의미를 담은 모델

컬렉션을 만들 때 size(차원)와 distance(거리 기준)를 반드시 지정한다

4. Vector DB

Vector DB — 수억 개 중에서 가까운 것을 빠르게

1

완전 탐색은 느리다

전부 하나씩 비교하면 수십만~수억 개에서 버틸 수 없다

2

ANN — 근사 최근접 탐색

조금의 정확도를 내주고 속도를 얻는 맛바꿈 — 「거의 정확하게, 아주 빠르게」

3

수천 건이면 완전 탐색으로 충분

파이썬 리스트로도 된다 — 그 위로는 전용 저장소

오늘 쓰는 것

Qdrant

REST·대시보드

관계형 DB 용어로 읽는 Qdrant

관계형 DB	Qdrant
테이블	컬렉션(collection)
행	포인트(point)
기본키	id
컬럼 값들	payload (JSON)
— (없음)	vector (숫자 배열)
인덱스	HNSW 등 ANN 인덱스

payload 에는 7-2에서 챙긴 metadata(source·page·chunk_id)를 그대로 넣는다 — 사람이 읽을 내용은 여기서 나온다

이 1시간에서 꼭 가져갈 것

- 1 임베딩은 텍스트를 고정 길이 숫자 배열로 바꾸는 것이다
- 2 차원은 모델이 정하고, 컬렉션의 size와 반드시 같아야 한다
- 3 같은 모델로 만든 벡터끼리만 비교할 수 있다
- 4 코사인 유사도는 방향만 보므로 문장 길이에 덜 흔들린다
- 5 벡터DB는 ANN으로 「거의 정확하게, 빠르게」 찾아 준다
- 6 payload에 원문과 출처를 넣어야 답변 근거를 밝힐 수 있다

7-3 · 이론 1H 마무리

다음은 임베딩 생성 실습 (1H)

임베딩을 직접 만들어 차원을 확인하고, 코사인 유사도를 손으로 계산해 비슷한 쌍과 다른 쌍을 비교합니다

✓ langchain-openai 또는 langchain-ollama 설치

✓ embed_query 로 차원 확인

✓ cos 함수로 유사도 직접 계산

✓ 문장 5개의 유사도 표와 배치 속도

준비할 것

API 키 또는 Ollama · 도커 실행 상태